

Segmentação de imagens utilizando o algoritmo de MST

Domynic Barros Lima   [Pontifícia Universidade Católica de Minas Gerais | Domynicbarroslima@gmail.com]

Kaiky França dos Reis   [Pontifícia Universidade Católica de Minas Gerais | kaikyfrs.comp@gmail.com]

Luís Fernando Rodrigues Braga   [Pontifícia Universidade Católica de Minas Gerais | luisbraga87@hotmail.com]

Mateus Ribeiro Fernandes   [Pontifícia Universidade Católica de Minas Gerais | mateus.fernandes.dev@gmail.com]

Mateus Resende Ottoni   [Pontifícia Universidade Católica de Minas Gerais | mateus.resende.ottoni@gmail.com]

Raquel de Parde Motta   [Pontifícia Universidade Católica de Minas Gerais | apxraquel@gmail.com]

 Instituto de Ciências Exatas e Informática, Pontifícia Universidade Católica de Minas Gerais, R. Dom José Gaspar, 500 - Coração Eucarístico, Belo Horizonte - MG, 30535-901, Brazil.

Received: 01 outubro 2025 • Accepted: DD Month YYYY • Published: 02 novembro 2025

Abstract. A segmentação de imagens em visão computacional se refere ao processo de dividir uma imagem em múltiplas regiões(conjunto de pixels) ou objetos, com o objetivo de simplificar ou modificar a representação de uma imagem para facilitar sua análise. Este trabalho propõe um método de segmentação baseado em MST para detecção de fronteiras entre regiões homogêneas, onde cada pixel é representado através de um vértice e as arestas são ponderadas a partir da intensidade da cor em relação aos seus pixels vizinhos. O algoritmo de MST então é aplicado para construir nossa AGM (Árvore geradora mínima) a partir de qual é definido limiares de cortes capazes de produzir segmentações consistente, preservado bordas relevantes. Também foram aplicadas técnicas para redução de ruído nas imagens, tornando o resultados mais eficientes.

Keywords: Segmentação de imagens; Algoritmo de Felzenszwalb e Huttenlocher; MST; Árvores geradoras mínimas; Processamento de imagens; AGM; Sobel; Gaussian Blur; Grafos.

1 Introdução

A segmentação de imagens é uma etapa fundamental em visão computacional e processamento de imagens, pois permite identificar e isolar regiões de interesse em uma imagem. Esse processo consiste em dividir uma imagem em múltiplas regiões Homogêneas, geralmente compostas por conjuntos de pixel, com características semelhantes, podendo ser elas como cor, textura, intensidade e etc. Por meio dessa divisão, buscamos simplificar a representação da imagem de modo a facilitar sua análise e interpretação por sistemas computacionais.

Diante da sua complexidade, o problema da segmentação tem sido amplamente estudado resultando em diversas soluções ao longo dos anos, soluções essas que incluem abordagens clássicas, como técnicas baseadas em limiares, detecção de bordas, crescimento de regiões entre outras. Já métodos mais recentes exploram representações por grafos e modelos de aprendizado profundo, capazes de lidar com estruturas mais complexas e variações sutis de cor e textura. Abordagens baseadas em grafo, se destacam por sua capacidade de representar relações locais entre os pixels da figura, permitindo que a segmentação seja formulada como um problema de partição em grafos ponderados. Essa representação favorece a detecção de fronteiras e a preservação de detalhes estruturais mais relevantes da imagem.

Neste trabalho, foi proposto um método de segmentação de imagens que combina pré-processamento com filtro [2]Gaussiano e detecção de bordas via [3]Sobel com um pós-processamento de fusão de superpixels, utilizando exclusivamente o algoritmo de [4]Felzenszwalb e Huttenlocher, tradicionalmente empregado para a construção de árvores gerado-

ras mínimas (MSTs). Nesse modelo, cada pixel da imagem é representado como um vértice, e as arestas conectam pixels vizinhos, com peso definido pela diferença de intensidade de cor entre eles. A partir da MST construída, o algoritmo aplica critérios de corte para identificar regiões homogêneas, garantindo segmentações consistentes e preservando bordas significativas. Além disso, técnicas de pré-processamento, como redução de ruído e conversão para imagens em tons de cinza, foram utilizadas para tornar a segmentação mais robusta e eficiente, independentemente do tipo de imagem ou da variação de cores presentes.



Figure 1. Exemplo de segmentação da gata Yuumi utilizando o algoritmo de Felzenszwalb e Huttenlocher

2 Metodologia Proposta

Conforme apresentado na introdução, o método proposto modela a imagem como um grafo não direcionado $G =$

(V, E, w) , onde cada pixel corresponde a um vértice e as arestas que conectam aos pixels vizinhos, considerando a conectividade de 1x8 (1 vértice que se liga aos 8 vizinhos).

Vale destacar que o cálculo da cor é feito através do modelo [1] **CIELAB** e não utilizando o padrão RGB, já que a formatação **CIELAB** é um modelo de cor tridimensional que representa as cores usando três eixos: L^* para luminosidade (0 = preto, 100 = branco), a^* para o eixo vermelho-verde ($+a^*$ é vermelho, $-a^*$ é verde) e b^* para o eixo amarelo-azul ($+b^*$ é amarelo, $-b^*$ é azul). Projetada para ser perceptualmente uniforme, permitindo a medição precisa das diferenças de cor assim como as percebidas pelo olho humano, se tornando assim o padrão mais comum de análise.

O peso da aresta é calculado com base a soma da diferença de cor (**padrão CIELAB**) entre os pixel adjacentes (Distância Euclidiana), valor do operador de Sobel é aplicado à vizinhança, que evidencia bordas, multiplicada por uma constante de ajuste. Dessa forma, quando uma aresta cruza uma borda perceptível, seu peso é automaticamente aumentado, dificultando a fusão de componentes que atravessam regiões de alto contraste. Esse mecanismo permite preservar fronteiras significativas enquanto regiões homogêneas são agrupadas de forma mais consistente e coerente com a imagem original.

$$w(e) = (\text{dist_cor}(p_i, p_j) + \text{Sobel}(p_i, p_j)) \cdot \alpha$$

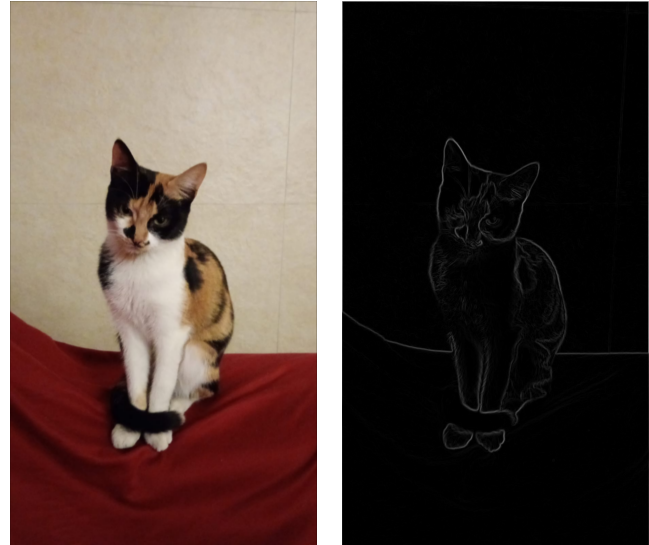
Com base na fórmula, temos o critério de fusão de nosso vértices, que é mecanismo central que é responsável por decidir quando dois componentes (ou segmentos) da imagem devem ser unidos. Para cada aresta do grafo, o algoritmo verifica seu peso $w(e)$ que conecta a dois componentes sendo eles C_1 e C_2 é suficientemente pequeno em relação a diferença interna de cada componente, onde dois componentes são unidos quando:

$$w(e) \leq \min \left(\text{Int}(C_1) + \tau(C_1), \text{Int}(C_2) + \tau(C_2) \right)$$

onde $\text{Int}(C)$ representa a diferença interna do componente C , definida como o maior peso de aresta presente em sua árvore geradora mínima (MST), e $\tau(C) = \frac{k}{|C|}$ é um limiar dependente do tamanho do componente e de uma constante k definida pelo usuário. Esse predicado garante que apenas componentes homogêneos e com características semelhantes sejam fundidos, enquanto bordas significativas e regiões distintas são preservadas, mantendo a consistência e a qualidade da segmentação final.

Antes da etapa de segmentação, é realizado um pré-processamento na imagem com o objetivo de reduzir ruídos, suavizar variações indesejadas e destacar bordas relevantes. Inicialmente, aplica-se o filtro Gaussian Blur, que suaviza a imagem através da convolução com uma função gaussiana, reduzindo ruídos de alta frequência e pequenas flutuações de intensidade. Esse processo garante que o cálculo dos pesos entre pixels vizinhos não seja afetado por irregularidades locais. Em seguida, utiliza-se o operador Sobel, responsável por realçar as bordas da imagem ao calcular o gradiente de intensidade em cada ponto. O resultado do Sobel é incorporado ao cálculo dos pesos das arestas do grafo, aumentando

automaticamente o valor de arestas que cruzam regiões de alto contraste. Assim, regiões homogêneas são preservadas, enquanto as fronteiras entre objetos são reforçadas, melhorando a consistência da segmentação final.



(a) Gata Lili com Gaussian Blur.

(b) Gata Lili com detecção de bordas via Sobel.

Figure 2. Comparação entre a imagem suavizada e o resultado da detecção de bordas.

A partir da aplicação dos processos de Pré-processamento, podemos dar continuidade a construção da nossa MST aplicando o pós-processamento em si, nesse caso a construção é feita a partir da construção incremental da MST, onde cada aresta é analisada em ordem crescente de peso. Para cada aresta, o algoritmo verifica se os componentes podem ou não serem fundidos em um super pixel.

2.1 Critério de Fusão de Componentes

Nesse caso, o algoritmo percorre todas as arestas da MST $e \in E$ na ordem definida. Onde para cada aresta C_1 e C_2 que conecte dois componentes distintos é avaliado o predicado de fusão:

$$w(e) \leq \min \left(\text{Int}(C_1) + \tau(C_1), \text{Int}(C_2) + \tau(C_2) \right)$$

onde:

- $\text{Int}(C)$ representa a **diferença interna** do componente C , definida como o maior peso de aresta presente em sua árvore geradora mínima (MST);
- $\tau(C) = \frac{k}{|C|}$ é um **limiar adaptativo**, dependente do tamanho do componente $|C|$ e de uma constante k definida pelo usuário.

O valor de k controla o grau de granularidade da segmentação:

- valores baixos de k produzem muitos segmentos pequenos, preservando detalhes;
- valores altos resultam em segmentações mais amplas e homogêneas.

Quando o predicado acima é satisfeito, os componentes C_1 e C_2 são fundidos em um novo componente C' , utilizando uma estrutura *Union-Find (disjoint-set)*. Essa estrutura mantém o rastreamento das uniões, permitindo que o algoritmo opere com complexidade quase linear $O(n \log n)$.

2.2 Pós-processamento e Fusão de Regiões

Após o término da fase inicial de fusão de vértices, o algoritmo pode gerar pequenos segmentos residuais formados por ruídos ou variações locais em certas regiões da imagem. Esses segmentos geralmente não representam objetos relevantes a serem avaliados, mas sim flutuações sutis de cor, textura ou até mesmo partes não muito bem identificadas causadas por apresentar limites pouco definidos entre objetos da própria imagem.

Para reduzir a ocorrência desses artefatos, é aplicada uma etapa de **pós-processamento**, na qual componentes com tamanho inferior a um valor mínimo predefinido (`min_size`) são unificados a componentes vizinhos maiores. Essa operação garante que não haja regiões isoladas ou deslocadas, melhorando a coerência visual e estrutural da segmentação final.

O procedimento é realizado percorrendo novamente todas as arestas do grafo, avaliando se os componentes conectados devem ser fundidos com base em seu tamanho e no peso das arestas correspondentes. Quando pelo menos um dos componentes possui tamanho inferior ao limite estabelecido, realiza-se a fusão:

se $(|C_a| < \text{min_size}) \vee (|C_b| < \text{min_size}) \Rightarrow \text{unir}(C_a, C_b)$

A implementação dessa etapa em C pode ser representada pelo código abaixo, onde ele inicia o processo de percorrer todas as arestas e verificar se há a necessidade de fusão dos pixels com base no valor encontrado para as suas respectivas arestas, caso não haja necessidade de realizar a fusão, o algoritmo apenas irá ignorar o conjunto de vértices e continuar sua busca até que todos tenham sido verificados.

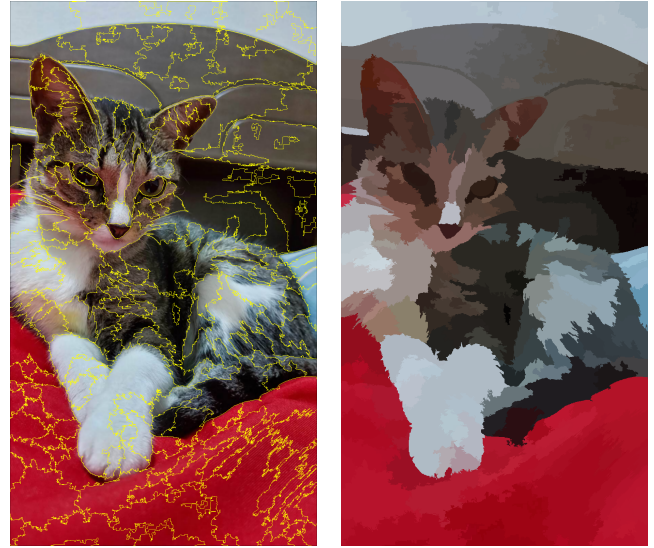
```
for (int i = 0; i < num_arestas; i++) {
    int a = encontrar(&grafo, arestas[i].v1);
    int b = encontrar(&grafo, arestas[i].v2);

    if (a != b && (tamanho[a] < min_size ||
        tamanho[b] < min_size)) {
        unir(&grafo, a, b, arestas[i].peso);
    }
}
```

Essa fase final do algoritmo atua como um *refinamento*, eliminando pequenos componentes residuais e suavizando a segmentação. Dessa forma, o resultado final apresenta regiões mais consistentes, sem comprometer a preservação das bordas e fronteiras detectadas na fase inicial como os exemplos a seguir.

3 Resultados e Discussão

Durante o processo de validação do método estudado, foram geradas 4 imagens a partir de cada etapa de tratamento e



(a) Gata Magali com a fusão e demarcação de bordas. (b) Gata Magali com Pós processamento já fundidos e coloridos.

Figure 3. Processo de pós processamento das imagens.

segmentação. Onde cada uma demonstra uma etapa realizada, sendo a variação de ruído, contraste, complexidade estrutural e a própria segmentação. O objetivo foi avaliar a robustez e eficiência do modelo em diferentes condições, observado a capacidade de preservação de bordas, agrupamento de regiões de modo homogêneo e redução de ruído local.

3.1 Pré-processamento

A etapa de pré-processamento apresentou resultados satisfatórios na suavização da imagem e na redução de ruídos de alta frequência. O filtro Gaussiano (Gaussian Blur), quando aplicado, eliminou flutuações locais de intensidade, gerando uma imagem mais uniforme e adequada para a segmentação. Essa etapa reduziu significativamente a interferência de ruídos, tornando o cálculo dos pesos entre pixels vizinhos mais estável e consistente para a aplicação do operador Sobel.

Em seguida, o operador Sobel destacou de forma mais nítida as fronteiras entre os objetos e áreas de transição de intensidade, criando uma imagem em preto e branco com as bordas mais bem definidas. Essa etapa é essencial para reforçar os pesos das arestas correspondente a região de alto contraste, evitando que a fusão incorreta de componente aconteça nas próximas etapas. Nas imagens analisadas, o contorno da borda ficou bem delimitado, com uma boa correspondência entre as transições reais da imagem.

3.2 Segmentação e Fusão de Regiões

Durante a fase de segmentação e fusão, o algoritmo de Felzenszwalb e Huttenlocher apresentou desempenho consistente, conseguindo identificar regiões visualmente coerentes e estruturalmente bem definidas. A utilização do modelo de cor CIELAB mostrou-se vantajosa já que permite um tratamento de cores mais coerente com a percepção humana, permitindo uma distinção mais precisa entre regiões com variações sutis de cor que poderiam ser menos perceptíveis caso fosse utilizado o padrão RGB para tratamento.

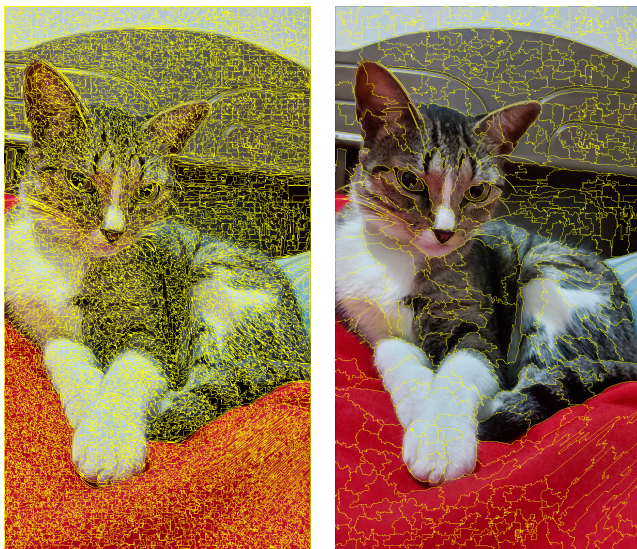
O critério de fusão baseado na diferença interna dos componentes e no parâmetro k controlou adequadamente o grau

da segmentação. Em valores intermediários de k , as imagens resultantes apresentaram regiões bem delimitadas, sem fragmentações excessivas. Já valores mais baixos tenderam a preservar detalhes finos, gerando um número maior de pequenos segmentos, enquanto valores mais altos promoveram fusões mais amplas, resultando em regiões maiores e mais homogêneas.

3.3 Pós-processamento e Consistência Visual

A etapa de pós processamento com a fusão dos super pixels gerados na etapa de segmentação anterior (um comportamento esperado comumente na segmentação de imagens) mostrou-se essencial para a qualidade e consistência visual do resultado final, tirando regiões de super pixels muito pequenos em pontos aleatórios da imagem e fundindo com regiões maiores e mais importantes, trazendo pixels mais consistentes, maiores e que fazem sentido para o contexto da imagem.

O parâmetro utilizado no algoritmo é o MIN_SEGMENT_SIZE, onde definimos um limiar mínimo de pixels que um segmento deve ter internamente para continuar existindo, caso contrário, será fundido com o segmento mais próximo.



(a) Gata Magali sem o pós processamento. (b) Gata Magali com Pós processamento.

Figure 4. Processo de pós processamento das imagens.

3.4 Teste comparativo e desempenho

Para a avaliação de desempenho, utilizou-se um notebook com as seguintes especificações: 12GB de memória RAM, CPU AMD Ryzen 5 4000 series e GPU integrada Radeon Vega 8.

A escalabilidade do algoritmo foi avaliada medindo-se o tempo de execução em imagens de resoluções variadas, de 10.000 (100x100) a 1.000.000 (1000x1000) pixels. A Tabela 1 apresenta os tempos absolutos em milissegundos, enquanto a Figura 5 plota esses dados.

Observa-se uma relação de crescimento quase linear entre o número de pixels (entrada) e o tempo de processa-

mento (custo). Um aumento de 100 vezes no número de pixels (de 10 mil para 1 milhão) resultou em um aumento de ≈ 110 vezes no tempo de execução. Este comportamento prático é consistente com a complexidade teórica esperada de $O(n \log n)$ do algoritmo.

Table 1. Tempo de execução (ms) versus Total de Pixels.

Total de Pixels	Tempo (ms)
10.000	89
40.000	360
90.000	821
160.000	1501
250.000	2324
360.000	3413
420.000	4745
640.000	6266
810.000	7938
1.000.000	9745

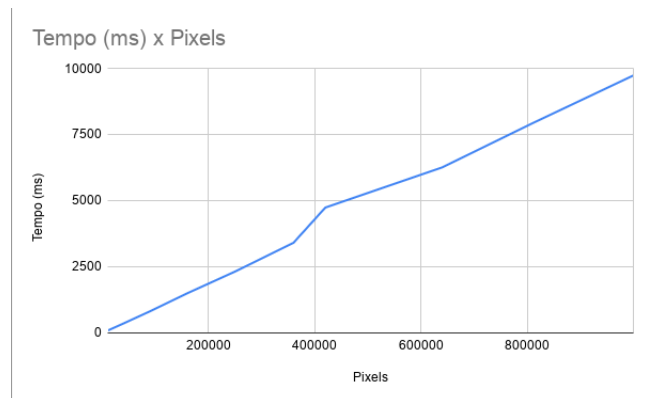


Figure 5. Relação entre o total de pixels (entrada) e o tempo de execução (ms).

4 Conclusão

Este trabalho apresentou um método de segmentação de imagens baseado no algoritmo de Felzenszwalb e Huttenlocher, aprimorado por etapas de pré-processamento, com técnicas de Gaussian Blur para suavização de imagens e detecção de bordas através do Sobel. Os resultados demonstram que o modelo é capaz de preservar fronteiras de maneira significativa e reduzir o impacto de ruídos, desde que hajam técnicas eficientes e capazes de processar a imagem e torna-la adequada para a segmentação, mantendo a coerência entre as regiões.

Adicionalmente, a análise quantitativa de desempenho confirmou a alta eficiência e escalabilidade do método implementado. A execução em imagens de resoluções variadas demonstrou que o tempo de processamento cresce de forma quase linear, um comportamento prático consistente com a complexidade teórica esperada de $O(n \log n)$ do algoritmo. Este resultado atesta a viabilidade da abordagem para aplicações que demandam o processamento de imagens em alta resolução.

Como perspectiva futura, há ideia de fazer uma investigação mais aprofundada de critérios adaptativos da fusão

baseados em aprendizado de máquina e aplicação de outras técnicas que visam tornar a segmentação ainda mais precisa e com um desempenho cada vez melhor, mesclando algoritmos e novas abordagens de remoção de ruído ou até mesmo de criação do grafo, podendo aumentar o escopo de aplicação para outras áreas, como em vídeos ou imagens em tempo real.

Declarations

Availability of data and materials

O código-fonte da implementação C++ descrita neste trabalho está disponível publicamente no seguinte repositório GitHub: <https://github.com/GLKaiky/ImageSegmentation>

References

- [1] T. F. Chan, C. K. M. Lee, and S. H. M. Ho. CIELAB-based color image segmentation. *Color research and application*, 25(1):49–57, 2000.
- [2] Richard O. Duda and Peter E. Hart. *Pattern classification and scene analysis*. Wiley, New York, 1973.
- [3] Ophelia L. N. Z. H. S. S. A. O. Fagundes, Claudio E. R. Pimentel, Marcelo d’Amorim, and Gustavo Callou. An efficient hardware implementation of the sobel operator. In *2011 VIII Southern Conference on Programmable Logic (SPL)*, pages 11–16, 2011.
- [4] Pedro F Felzenszwalb and Daniel P Huttenlocher. Efficient graph-based image segmentation. *International journal of computer vision*, 59(2):167–181, 2004.